

23. State Design Pattern in Java

23.1 Introduction

The **State Design Pattern** is a **behavioral design pattern** that allows an object to alter its behavior when its internal state changes, appearing as if it changed its class.

23.2 Problem it Solves

Helps avoid complex if-else or switch statements by encapsulating state-specific behaviors into separate classes.

23.3 What is Observer Pattern?

It consists of:

- **Context:** The object whose behavior changes
- **State:** An interface for declaring state-specific behavior
- **Concrete States:** Classes that implement the state-specific behavior

23.4 Real-World Analogy

A **fan** that can be in "Off", "Low", "Medium", or "High" speed state. Each button press changes the fan's behavior based on its current state.

23.5 Structure

```
package com.mannemlokes;

public interface State {
    void handle();
}

class Context {
    private State currentState;

    public void setState(State state) {
        this.currentState = state;
    }

    public void request() {
        currentState.handle();
    }
}
```

23.6 Benefits

- Removes conditional logic
- Improves maintainability and readability
- Enables state transitions at runtime

23.7 Implementation Steps

1. Create a State interface
2. Implement concrete states
3. Create a Context to manage state and behavior

23.8 Examples

Example1: Fan Speed Control

```
package com.mannemlokes.example1;

interface FanState {
    void pressButton(Fan fan);
}

class Fan {
    private FanState state;

    public Fan() {
        state = new OffState();
    }

    public void setState(FanState state) {
        this.state = state;
    }

    public void pressButton() {
        state.pressButton(this);
    }
}

class OffState implements FanState {
    public void pressButton(Fan fan) {
        System.out.println("Turning fan to low speed.");
        fan.setState(new LowSpeedState());
    }
}

class LowSpeedState implements FanState {
    public void pressButton(Fan fan) {
        System.out.println("Turning fan to high speed.");
        fan.setState(new HighSpeedState());
    }
}

class HighSpeedState implements FanState {
    public void pressButton(Fan fan) {
        System.out.println("Turning fan off.");
        fan.setState(new OffState());
    }
}

public class Example1FanClient {
    public static void main(String[] args) {
        Fan fan = new Fan();
        fan.pressButton(); // Low
        fan.pressButton(); // High
        fan.pressButton(); // Off
    }
}
```

Output:

```
Turning fan to low speed.
Turning fan to high speed.
Turning fan off.
```

Example2: Vending Machine

```
package com.mannemlokes.example2;

interface VendingState {
    void insertCoin(VendingMachine machine);

    void dispense(VendingMachine machine);
}

class VendingMachine {
    private VendingState state;

    public VendingMachine() {
        state = new NoCoinState();
    }

    public void setState(VendingState state) {
        this.state = state;
    }

    public void insertCoin() {
        state.insertCoin(this);
    }

    public void dispense() {
        state.dispense(this);
    }
}

class NoCoinState implements VendingState {
    public void insertCoin(VendingMachine machine) {
        System.out.println("Coin inserted.");
        machine.setState(new HasCoinState());
    }

    public void dispense(VendingMachine machine) {
        System.out.println("Insert coin first.");
    }
}

class HasCoinState implements VendingState {
    public void insertCoin(VendingMachine machine) {
        System.out.println("Already have a coin.");
    }

    public void dispense(VendingMachine machine) {
        System.out.println("Item dispensed.");
        machine.setState(new NoCoinState());
    }
}

public class Example2VendingClient {
    public static void main(String[] args) {
        VendingMachine vm = new VendingMachine();
        vm.dispense(); // Insert coin first
        vm.insertCoin(); // Coin inserted
        vm.dispense(); // Item dispensed
    }
}
```

Output:

```
Insert coin first.
Coin inserted.
Item dispensed.
```

Example3: TCP Connection States

```
package com.mannemlokes.example3;

interface TCPState {
    void open(TCPConnection connection);

    void close(TCPConnection connection);
}

class TCPConnection {
    private TCPState state;

    public TCPConnection() {
        state = new ClosedState();
    }

    public void setState(TCPState state) {
        this.state = state;
    }

    public void open() {
        state.open(this);
    }

    public void close() {
        state.close(this);
    }
}

class ClosedState implements TCPState {
    public void open(TCPConnection connection) {
        System.out.println("Opening connection.");
        connection.setState(new OpenState());
    }

    public void close(TCPConnection connection) {
        System.out.println("Connection already closed.");
    }
}

class OpenState implements TCPState {
    public void open(TCPConnection connection) {
        System.out.println("Connection already open.");
    }

    public void close(TCPConnection connection) {
        System.out.println("Closing connection.");
        connection.setState(new ClosedState());
    }
}

public class Example3TCPClient {
```

```

public static void main(String[] args) {
    TCPConnection conn = new TCPConnection();
    conn.open(); // Opening
    conn.open(); // Already open
    conn.close(); // Closing
    conn.close(); // Already closed
}
}

```

Output:

```

Opening connection.
Connection already open.
Closing connection.
Connection already closed.

```

23.9 State vs Strategy vs Command

Pattern	Intent
State	Change behavior based on internal state
Strategy	Select algorithm at runtime
Command	Encapsulate a request as an object

23.10 Pros and Cons

✓ Pros

- Eliminates complex if-else conditions
- Encapsulates each state logic separately
- Easy to add new states

✗ Cons

- More classes to manage
- Can introduce complexity for simple scenarios

23.11 When to Use / Not to Use

Use When:

- Object behavior depends on its state
- You want to avoid long if-else/switch

Avoid When:

- Few states and logic is simple

23.12 Summary

Feature	Description
Intent	Change behavior based on internal state
Use Case	UI navigation, lifecycle, parsers
Key Benefit	Cleaner code and better maintainability

The **State Pattern** is ideal when objects need to change behavior dynamically depending on their internal state.